

Deep Learning: Homework #3

Due: March 6, 2026 at 11:59 pm

Total Points: 47

Neural Network Architecture

Consider the following neural network for predicting $\mathbf{y}$, a one-hot encoding of $K = 5$ categories:

$$\mathbb{E}[\mathbf{y}|\mathbf{X}] = \mathbf{W}_2^T \mathbf{h}_2$$

$$\mathbf{h}_2 = \text{ReLU}(\tilde{\mathbf{a}}_2) + \mathbf{h}_1$$

$$\tilde{\mathbf{a}}_2 = \text{LayerNorm}(\mathbf{a}_2)$$

$$\mathbf{a}_2 = \mathbf{W}_1^T \mathbf{h}_1$$

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{a}_1)$$

$$\mathbf{a}_1 = \mathbf{W}_0^T \mathbf{x}$$

where $\mathbf{x} \in \mathbb{R}^{100}$ and $\mathbf{W}_0 \in \mathbb{R}^{100 \times 10}$. For simplicity, **assume that there are no bias / intercept parameters in the fully-connected transformations.**

Problem 1

Answer the following questions about the above-defined architecture.

- (a) How many total hidden units does this architecture have?

In other words, what is the sum of the dimensions of $\mathbf{h}_1$ and $\mathbf{h}_2$? **[3 Points]**

$$\mathbf{x} \in \mathbb{R}^{100}, \mathbf{W}_0 \in \mathbb{R}^{100 \times 10}, \therefore \mathbf{h}_1 / \mathbf{a}_1 \in \mathbb{R}^{10}$$

So the dimensions of $\mathbf{h}_1$ is 10.

$$\mathbf{a}_2 = \mathbf{W}_1^T \mathbf{h}_1 \text{ and } \mathbf{h}_2 = \text{ReLU}(\tilde{\mathbf{a}}_2) + \mathbf{h}_1$$

To make them able to calculate, $\mathbf{W}_1 \in \mathbb{R}^{10 \times 10}$, So $\mathbf{h}_2 \in \mathbb{R}^{10}$

The dimensions of $\mathbf{h}_2$ is 10.

So the total hidden units = $10 + 10 = 20$.

- (b) How many total trainable parameters does this architecture have? That is, how many parameters would be updated by gradient descent? Note that we have assumed there are no bias / intercept terms. [3 Points]

$$\mathbb{E}[\mathbf{y}|\mathbf{X}] = W_2^T h_2, \text{ output } K=5, \therefore W_2 \in \mathbb{R}^{10 \times 5}$$

In each gradient descent iteration, all weights W_0, W_1, W_2 should be updated.

$$N = 100 \times 10 + 10 \times 10 + 10 \times 5 = 1150$$

So 1150 parameters are trainable.

- (c) When applying auto-differentiation (auto-diff) to this architecture, would there be an advantage to using either forward mode or reverse model auto-diff? If so, which (forward or reverse) would be better? [3 Points]

Forward mode is efficient when input dimension is small and reverse mode is efficient when output dimension is small, it is also standard for neural network training.

In this case, Reverse mode should be better, because reverse mode calculate from the deeper(output) layer to the shallower(input) layer. Because the output is 5, which is much smaller than the input=100. So it would be better to start from the output layer, that's why we choose reverse mode.

Batch Normalization vs Layer Normalization

Consider the differences between *Batch Normalization* (BatchNorm) and *Layer Normalization* (LayerNorm). Assume we have a matrix of pre-activations $\mathbf{A} \in \mathbb{R}^{N \times D}$, with N being the number of instances / data points and D being the number of dimensions at this hidden layer. BatchNorm applies the following normalization to each element $a_{n,d}$ of $\mathbf{A}$:

$$\tilde{a}_{n,d} = \text{BatchNorm}(a_{n,d}; \mathbf{A}, \gamma_d, \beta_d) = \beta_d + \gamma_d \cdot \frac{a_{n,d} - \mathbb{E}[a_{\cdot,d}]}{\sqrt{\text{Var}[a_{\cdot,d}] + \epsilon}}$$

where $\beta_d \in \mathbb{R}$ and $\gamma_d \in \mathbb{R}$, $d \in [1, D]$, are per-dimension parameters to be learned, $\epsilon \in \mathbb{R}^+$ is a small positive constant for numerical stability, $\mathbb{E}[a_{\cdot,d}]$ is the mean (first moment) of the d -th dimension as computed empirically over the N samples contained within $\mathbf{A}$, and

$\text{Var}[\mathbf{a}_{:,d}]$ is the variance (second moment) of the d -th dimension as computed empirically over the N samples contained within $\mathbf{A}$. In other words, BatchNorm performs its normalization by computing the mean and variance statistics for each column of $\mathbf{A}$.

LayerNorm, on the other hand, computes the mean and variance parameters across *the rows of* $\mathbf{A}$. Its transformation can be written as:

$$\bar{\mathbf{a}}_{n,d} = \text{LayerNorm}(\mathbf{a}_{n,d}; \mathbf{a}_n, \gamma_d, \beta_d) = \beta_d + \gamma_d \cdot \frac{a_{n,d} - \mathbb{E}[\mathbf{a}_{n,:}]}{\sqrt{\text{Var}[\mathbf{a}_{n,:}] + \epsilon}},$$

where $\beta_d \in \mathbb{R}$ and $\gamma_d \in \mathbb{R}$, $d \in [1, D]$, are per-dimension parameters to be learned, $\epsilon \in \mathbb{R}^+$ is (again) a small positive constant for numerical stability, $\mathbb{E}[\mathbf{a}_{n,:}]$ is the mean (first moment) of the n pre-activation vector as computed empirically over the D dimensions contained within $\mathbf{a}_n$, and $\text{Var}[\mathbf{a}_{n,:}]$ is the variance (second moment) of the n -th per-activation vector as computed empirically over the D dimensions contained within $\mathbf{a}_n$. In other words, LayerNorm performs its normalization by computing the mean and variance statistics for each *row* of $\mathbf{A}$.

Problem 2

Consider the following simple NN with two hidden units for real-valued regression:

$$\mathbb{E}[y|x] = w_2 \cdot h_0 + w_3 \cdot h_1,$$

$$\mathbf{h} = [h_0, h_1] = \text{Logistic} \circ \text{Norm}([w_0 \cdot x, w_1 \cdot x]),$$

where $w_0, w_1, w_2, w_3 \in \mathbb{R}$ are the weight parameters and

$$\text{Logistic} \circ \text{Norm}([a_0, a_1])$$

means that we apply some normalization function (**Norm**), such as BatchNorm or LayerNorm, and then the logistic activation function. Assume that we would train this model with the squared loss function.

- (a) Assume the normalization function is *BatchNorm*, and the training set is of size N . Derive an expression for the derivative of the loss with respect to w_0 . [6 Points]
let $E[y | x] = \hat{y}$

$$\frac{\partial l}{\partial w_0} = \frac{\partial l}{\partial \hat{y}} \left(\frac{\partial \hat{y}}{\partial h_0} \frac{\partial h_0}{\partial w_0} + \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial w_0} \right)$$

because we use squared loss function: $l = \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n)^2$

$$\frac{\partial l}{\partial \hat{y}} = 2(\hat{y} - y)$$

And we can easliy calculate:

$$\frac{\partial \hat{y}}{\partial h_0} = w_2, \quad \frac{\partial \hat{y}}{\partial h_1} = w_3$$

According to the problem, $h = \text{Logistic} \circ \text{Norm}([w_0 \cdot x, w_1 \cdot x])$,

while using BatchNorm, each dimension only calculate in itself, it means h_1 contains no w_0 ,

$$\text{So } \frac{\partial h_1}{\partial w_0} = 0$$

$$\therefore a_0 = w_0 x, \therefore \frac{\partial h_0}{\partial w_0} = \frac{\partial h_0}{\partial a_0} x$$

We know that $h_0 = \text{Logistic} \circ \text{BatchNorm}([a_0])$

With regard to BatchNorm:

$$\mu = \frac{1}{N} \sum_{i=1}^N a_{i,0}, \quad var = \frac{1}{N} \sum_{i=1}^N (a_{i,0} - \mu)^2$$

$$\text{BatchNorm}[a_0] = \hat{a}_n = \beta_0 + \gamma_0 \frac{a_{n,0} - \mu}{\sqrt{var + \epsilon}}$$

$$\therefore \frac{\partial h_0}{\partial a_{n,0}} x_n = x_n \frac{\partial h_0}{\partial \hat{a}_n} \frac{\partial \hat{a}_n}{\partial a_{n,0}} = x h_0 (1 - h_0) \gamma_0 \frac{\partial \frac{a_{n,0} - \mu}{\sqrt{var + \epsilon}}}{\partial a_{n,0}}$$

$$= x h_0 (1 - h_0) \gamma_0 \left[\frac{N-1}{N} \frac{1}{\sqrt{var + \epsilon}} - \frac{(a_0 - \mu)^2}{N(var + \epsilon)^{3/2}} \right]$$

So we can get:

$$\frac{\partial l}{\partial w_0} = 2(E[y | x] - y)(w_2 x h_0 (1 - h_0) \gamma_0 \left[\frac{N-1}{N} \frac{1}{\sqrt{var + \epsilon}} - \frac{(a_0 - \mu)^2}{N(var + \epsilon)^{3/2}} \right])$$

- (b) Now assume the normalization function is *LayerNorm*, and the training set is of size N . Derive an expression for the derivative of the loss with respect to w_0 . [6 Points]

When change to LayerNorm, everything is the same as a) except now

$$\frac{\partial h_1}{\partial w_0} \neq 0 \text{ because it will combine both dimensions}$$

$$\mu_n = \frac{1}{2}(a_{n,0} + a_{n,1}), \quad var_n = \frac{1}{2} \sum_{d=0}^1 (a_{n,d} - \mu_d)^2$$

Similarly, we can get:

$$\frac{\partial h_0}{\partial w_0} = x h_0 (1 - h_0) \gamma_0 \frac{\partial \frac{a_{n,0} - \mu_n}{\sqrt{var_n + \epsilon}}}{\partial a_{n,0}}$$

$$\frac{\partial h_1}{\partial w_0} = x h_1 (1 - h_1) \gamma_0 \frac{\partial \frac{a_{n,1} - \mu_n}{\sqrt{var_n + \epsilon}}}{\partial a_{n,0}}$$

$$\text{Let } \frac{\partial \frac{a_{n,d} - \mu_n}{\sqrt{var_n + \epsilon}}}{\partial a_{n,0}} = \tilde{a}_{n,d}$$

$$\text{We can get: } \frac{\partial l}{\partial w_0} = 2(E[y | x] - y)(w_2 x h_0 (1 - h_0) \gamma_0 \tilde{a}_{n,0} + w_3 x h_1 (1 - h_1) \gamma_0 \tilde{a}_{n,1})$$

- (c) Describe the differences between the final derivatives from parts (a) and (b), tracing the differences to the mechanisms by which LayerNorm and BatchNorm operate. [2 Points]

LayerNorm has an additional term: $w_3 x h_1 (1 - h_1) \gamma_0 \tilde{a}_{n,1}$, And the calculation method of μ and var in each Norm is also different.

This differences is because that BatchNorm only calculates expectation and Variance in one dimension(one column), so h_1 will not contain a_0 or w_0 , while LayerNorm is a cross-dimension Normalization so h_1 will rely on the value of both a_0 and a_1 .

Second Derivatives and Adaptive Learning Rates

We usually train a neural network via gradient descent, i.e. :

$$\mathbf{W}_l^{t+1} = \mathbf{W}_l^t - \alpha \cdot \nabla_{\mathbf{W}_l^t} \ell(\mathbf{W}_0^t, \dots, \mathbf{W}_L^t; \mathcal{D})$$

where $\mathbf{W}_l^t$ are the parameters at layer L at iteration t and $\alpha \in \mathbb{R}$ is the step size (i.e. learning rate). This is called a *first-order* optimization method since the gradient vector is comprised of first derivatives. However, there are *second-order* optimization methods that use the update:

$$\mathbf{W}_l^{t+1} = \mathbf{W}_l^t - \mathcal{H}_{\mathbf{W}_l^t}^{-1} \nabla_{\mathbf{W}_l^t} \ell(\mathbf{W}_0^t, \dots, \mathbf{W}_L^t; \mathcal{D})$$

where $\mathcal{H}_{\mathbf{W}_l^t}^{-1}$ is the inverse of the Hessian matrix evaluated at $\mathbf{W}_l^t$. The Hessian matrix is the matrix of all second derivatives. That is, for a scalar function $f(\mathbf{x})$ and $\mathbf{x} \in \mathbb{R}^D$, the Hessian matrix is of size $(D \times D)$ and is defined as:

$$\mathcal{H} = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_D} \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_2^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_D} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_D \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_D \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_D^2} \end{bmatrix}.$$

While we don't usually use a second-order method for neural networks since computing, storing, and inverting the Hessian is too

costly, it is still useful to think of the structure of the Hessian for particular architecture choices. We want the Hessian matrix to be ‘close’ to diagonal—meaning that all the cross derivatives in the off-diagonal elements are near zero—because then each parameter update does not significantly depend on the value of the other parameters. This makes the optimization problem ‘easier’ for first-order methods since the parameters act independently instead of being highly dependent on each other’s values.

Problem 3

Consider the following simple NN for real-valued regression. It has just three parameters:

$$\mathbb{E}[y|x] = w_2 \cdot h, \quad h = \phi(w_0 \cdot x_0 + w_1 \cdot x_1),$$

where $w_0, w_1, w_2 \in \mathbb{R}$ are the parameters and ϕ is the activation function. Assume the data set just has one data point, $\mathcal{D} = (\mathbf{x} \in \mathbb{R}^2, y \in \mathbb{R})$, to simplify and de-clutter the derivations below.

- (a) Derive the 2×2 Hessian matrix for the parameters in the first layer (w_0 and w_1) when the activation function is the logistic function. Hint: the cross derivatives $\partial^2 \ell / (\partial w_0 \partial w_1)$ and $\partial^2 \ell / (\partial w_1 \partial w_0)$ should be equal, so you only need to derive three expressions, not four. [9 Points]

let $a = w_0 x_0 + w_1 x_1$, $\hat{y} = w_2 h$

$$\frac{\partial \ell}{\partial w_0} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial w_0} = (\hat{y} - y) w_2 h (1 - h) x_0$$

$$\frac{\partial \ell}{\partial w_1} = (\hat{y} - y) w_2 h (1 - h) x_1$$

So we can get:

$$\begin{aligned} \frac{\partial^2 \ell}{\partial^2 w_0} &= \frac{\partial \hat{y} - y}{\partial w_0} w_2 h (1 - h) x_0 + (\hat{y} - y) \frac{\partial w_2 h (1 - h) x_0}{\partial w_0} = \frac{\partial a}{\partial w_0} w_2^2 h^2 (1 - h)^2 x_0^2 \\ &+ (\hat{y} - y) w_2 x_0 (1 - 2h) h (1 - h) \frac{\partial a}{\partial w_0} = w_2^2 h^2 (1 - h)^2 x_0^2 + (\hat{y} - y) w_2 x_0 (1 - 2h) h (1 - h) x_0 \\ &= x_0^2 [w_2^2 h^2 (1 - h)^2 + (\hat{y} - y) w_2 h (1 - h) (1 - 2h)] \end{aligned}$$

$$\text{similarly, } \frac{\partial^2 \ell}{\partial^2 w_1} = x_1^2 [w_2^2 h^2 (1 - h)^2 + (\hat{y} - y) w_2 h (1 - h) (1 - 2h)]$$

$$\frac{\partial^2 \ell}{\partial w_0 \partial w_1} = x_0 x_1 [w_2^2 h^2 (1 - h)^2 + (\hat{y} - y) w_2 h (1 - h) (1 - 2h)]$$

- (b) Derive the 2×2 Hessian matrix for the parameters in the first layer (w_0 and w_1) when the activation function is the ReLU activation. Hint: the cross derivatives $\partial^2 \ell / (\partial w_0 \partial w_1)$

and $\partial^2 \ell / (\partial w_1 \partial w_0)$ should be equal, so you only need to derive three expressions, not four. [9 Points]

$$h = \text{ReLU}(a) = \max(0, a)$$

$$\text{So } \phi'(a) = \frac{\partial h}{\partial a} = \begin{cases} 0 & \text{if } a \leq 0 \\ 1 & \text{if } a > 0 \end{cases}$$

$$\text{And } \phi''(a) = \frac{\partial^2 h}{\partial^2 a} = 0$$

Also first calculate:

$$\begin{aligned} \frac{\partial l}{\partial w_0} &= \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial w_0} = (\hat{y} - y) w_2 \phi'(a) x_0 \\ \frac{\partial l}{\partial w_1} &= (\hat{y} - y) w_2 \phi'(a) x_1 \end{aligned}$$

Then we can get:

$$\begin{aligned} \frac{\partial^2 l}{\partial^2 w_0} &= \frac{\partial(\hat{y} - y)}{\partial w_0} w_2 \phi'(a) x_0 + (\hat{y} - y) \frac{\partial w_2 \phi'(a) x_0}{\partial w_0} = w_2^2 \phi'^2(a) x_0^2 + \\ &(\hat{y} - y) w_2 x_0 \phi'' x_0 = w_2^2 \phi'^2(a) x_0^2 \end{aligned}$$

Similarly, we can calculate:

$$\begin{aligned} \frac{\partial^2 l}{\partial^2 w_1} &= w_2^2 \phi'^2(a) x_1^2 \\ \frac{\partial^2 l}{\partial^2 w_0 \partial w_1} &= w_2^2 \phi'^2(a) x_0 x_1 \end{aligned}$$

So when $a \leq 0$, $\phi'(a) = 0$, each cross derivatives both equal to 0, and when $a > 0$, $\phi'(a) = 1$,

$$\begin{aligned} \frac{\partial^2 l}{\partial^2 w_1} &= w_2^2 x_1^2 \\ \frac{\partial^2 l}{\partial^2 w_0} &= w_2^2 x_0^2 \\ \frac{\partial^2 l}{\partial w_0 \partial w_1} &= \frac{\partial^2 l}{\partial w_1 \partial w_0} = w_2^2 x_0 x_1 \end{aligned}$$

- (c) Using your results from (a) and (b), for which activation function do we expect a first-order gradient method to perform better and why? [2 Points]

From (a) and (b), we can summarize:

$$H_{\text{logistic}} = [w_2^2 h^2 (1-h)^2 + (\hat{y} - y) w_2 h (1-h) (1-2h)] \begin{bmatrix} x_0^2 & x_0 x_1 \\ x_0 x_1 & x_1^2 \end{bmatrix}$$

$$H_{\text{ReLU}} = \begin{cases} 0 & \text{if } a \leq 0 \\ w_2^2 \begin{bmatrix} x_0^2 & x_0 x_1 \\ x_0 x_1 & x_1^2 \end{bmatrix} & \text{if } a > 0 \end{cases}$$

ReLU activation function is better because it is linear and

induces a simpler and more stable structure in the Hessian matrix, which do not contain h (input parameters).

- (d) Notice that in the 2nd-order gradient update, the inverse Hessian matrix is in the same place in the equation as the step-size usually is. In fact, there is no step size parameter in this equation since conditioning by the inverse Hessian can be thought of as having an *automatic, adaptive step size*. Explain why multiplying by the inverse Hessian automatically adjusts the step size in a beneficial way. [2 Points]

Because the Hessian matrix is changeable during the training process, it contains w_2^2 in its formula, so with the training going on, the weight becomes smaller and therefore Hessian matrix becomes smaller too, and its inverse becomes larger, which controls the speed of training. As a result, large curvature leads to smaller steps, and small curvature leads to larger steps. In this way, the inverse Hessian acts as an adaptive step size, adjusting automatically during training.

- (e) Recall that the *Adam optimizer* uses an adaptive step size of $\alpha/(\sqrt{\hat{v}_t} + \epsilon)$ where $\alpha \in \mathbb{R}^+$ is a fixed step size, $\hat{v}_t$ is a running estimate of the gradient's variance at time t , and $\epsilon \in \mathbb{R}^+$ is a small constant just for numerical stability. Do you see any similarities between how Adam sets its adaptive step size and setting the step size according to the inverse Hessian? [2 Points]

The similarity is that both Adam and the inverse Hessian rescale the gradient according to curvature, Hessian matrix uses the inverse Hessian, while Adam approximates curvature using gradient variance. With the weight become smaller, its curvature becomes small, so its variance becomes smaller, then $\alpha/\sqrt{\text{var}} + \epsilon$ becomes larger, which makes the step size becomes larger, it is the same as we described in (d).